Typechecking of Overloading

in Programming Languages and Mechanized Mathematics

Arthur Charguéraud
with Martin Bodin, Jana Dunfield, and Louis Riboulet

Inria

Journées LVP, Nov. 13th, 2025

Mathematics: overloaded symbols

$$x + y \qquad \text{vs} \qquad x +_{\mathbb{Z}} y$$

$$\sum_{d \in \{i, 2i\}} \sum_{k \in -6..7} 3 \cdot e^{\frac{d \cdot \pi}{8}} \cdot M^{2 \cdot k^2} \cdot N$$

Three centuries of mathematics...
but where are the typing rules for overloading resolution?

Resolution of overloading is intertwined with typechecking:

- overloading resolution depends on types
- types of overloaded symbols depend on resolution.

Work on static resolution of overloading

- C++: overloading resolution
  → but guided only by arguments, never by the context
  → in particular, no overloading for constants
- Rocq's notation scope
  → guided ony by the context, not by the argument
- PVS, ADA: resolution guided by arguments and context
  → but no type inference: all bindings must be annotated
- Haskell, Rocq: typeclasses
  → but with exponential complexity

This work: first polynomial-time algorithm for overloading resolution, guided both by arguments and by the context, and with local type inference.

Problem formulation

Instances description: to every symbol is associated a list of candidate instances, each with a type and a value.

Input program: code with occurrences of overloaded symbols.
Extracted program: symbols are replaced with the selected values.

Goal: unambiguous resolution, i.e. an instance can be selected iff it is the only one that leads to a well-typed extracted program.

Desirable properties: efficiency, predictability, helpful errors.

Simplifying hypotheses:

- explicit annotation for polymorphic functions and let-bindings
- no ML-style partial applications

NP-hardness: encoding to 3-SAT

$$(x_1 \vee x_3 \vee \neg\, x_4) \;\wedge\; (x_1 \vee x_4 \vee \neg\, x_5) \;\wedge\; (x_2 \vee \neg\, x_3 \vee x_5) \;\wedge\; (\neg\, x_2 \vee x_3 \vee \neg\, x_5)$$

```
__instance 0 : int (* true and false *)
__instance 0 : float
__instance neg : float -> int (* negation *)
__instance neg : int -> float
__instance f : int -> float -> float -> unit (* at least one true *)
__instance f : float -> int -> float -> unit
__instance f : float -> float -> int -> unit
__instance f : int -> int -> float -> unit
__instance f : int -> float -> int -> unit
__instance f : float -> int -> int -> unit
__instance f : int -> int -> int -> unit
let x1 = 0 in (* int or float *)
let x2 = 0 in
let x3 = 0 in
let x4 = 0 in
let x5 = 0 in
f x1 x3 (neg x4); (* at least one argument must be int *)
f x1 x4 (neg x5);
f x2 (neg x3) x5;
f (neg x2) x3 (neg x6);
```

Iteratively solvable programs

Material written programmers and mathematicians are meant to be easily interpretable, not to serve as Sudoku-style puzzles.

Consider only programs that are *iteratively solvable* by a sequence of simple deduction steps of the form:

1. The expressions t_1, .., t_n have type T_1, .., T_n, hence the expression t_r has type T_r.
2. The overloaded symbol x must have a type of the form T; among the list of candidate instances for x, only the instance I has a type that unifies with T, hence x resolves to I.

Nonlocal deduction flow

```
__instance 0 : int
__instance 0 : float
__instance f : int -> int -> unit
__instance f : float -> float -> unit
__instance f : char -> bool -> unit

let x0 : int = 0 in
let x1 = 0 in
let x2 = 0 in
let x3 = 0 in
let x4 = 0 in
let x5 = 0 in
let x6 = 0 in
let x7 = 0 in
let x8 = 0 in
f x7 x8;
f x5 x6;
f x3 x4;
f x1 x2;
f x0 x1;
f x2 x3;
f x4 x5;
f x6 x7;
```

Overview of the proposed algorithm

Typechecking algorithm:

1. Unify types like in standard ML typechecking
2. Collect overloaded symbols in traversal order
3. Iteratively try to resolve overloaded symbols

Resolution rule:
Assume symbol x with a type T constrained by the context.
Assume x has candidate instances $(v_1 : T_1)$, ..., $(v_n : T_n)$.
Symbol x resolves to v_i iff T unifies with T_i but not with T_j for $j \neq i$.

Representation of terms and types

In this talk, omitting details related to polymorphism.

Annotated term	t	$::=$	$u^{:T}$
Contents of a term	u	$::=$	$x_{\mathsf{id}} \mid v \mid t_1(t_2) \mid \mathsf{let}\, x^{:T} = t_1\, \mathsf{in}\, t_2 \mid \lambda x^{:T}.\, t_1$
Typing environment	E	$::=$	$\emptyset \mid E,\, x : \mathsf{Regular}(T) \mid E,\, x : \mathsf{Overloaded}(I)$
Set of instances	I	$::=$	$\emptyset \mid I, (v : T)$
Type representation	T	$::=$	*unique identifiers*
Type description	D	$::=$	$\mathsf{Flexible} \mid \mathsf{Unified}\, T \mid \mathsf{Constr}(C, \vec{T})$
Mutable state	s	$::=$	$\emptyset \mid s[T := D] \mid s[\mathsf{id} := v] \mid s[\mathsf{id} := (T, I)]$

Initialization: annotate each AST node with a fresh, unconstrained type.
Unifications: the operation unify(T_1, T_2) refines the types.

Implementation of unification—standard!

```
type id = unit ref

type typ = desc ref

and desc =
  | Flexible
  | Unified of typ
  | Constr of id * typ list

let rec unify (t1:typ) (t2:typ) : unit =
  if !t1 != !t2 then
  match !t1, !t2 with
  | Unified t1', _ -> unify t1' t2
  | _, Unified t2' -> unify t1 t2'
  | Flexible, _ -> t1 := Unified t2
  | _, Flexible -> t2 := Unified t1
  | Constr(c1,ts1), Constr(c2,ts2) ->
      if c1 != c2 || List.length ts1 <> List.length ts2 then raise Failure;
      List.iter2 unify ts1 ts2
```

Omitted: union-find path compression and union-by-rank optimizations.

Handling term constraints

Subterm labelled with its type	Operations to apply
$(\mathsf{let}\, x^{:T_0} = u_1{}^{:T_1}\, \mathsf{in}\, u_2{}^{:T_2})^{:T}$	$\mathsf{unify}(T_0, T_1)\,;\, \mathsf{unify}(T_2, T)$
$(u_0{}^{:T_0}(u_1{}^{:T_1}))^{:T}$	$\mathsf{unify}(T_0,\, T_1 \to T)$
$(\lambda x^{:T_0}.\, u_1{}^{:T_1})^{:T}$	$\mathsf{unify}(T,\, T_0 \to T_1)$
$x^{:T}_{\mathsf{id}}$ if x is bound to $\mathsf{Regular}(T')$	$\mathsf{unify}(T', T)$
$x^{:T}_{\mathsf{id}}$ if x is bound to $\mathsf{Overloaded}(I)$	$s' := s[\mathsf{id} := (T, I)]$

Likewise for functions involving more than one arguments.

Details of symbol resolution—backtracking involved!

Consider an occurrence $x^{:T}_{\mathsf{id}}$ of a not-yet-resolved overloaded symbol.

$$s[\mathsf{id}] = (T, I) \quad \text{where} \quad I = (v_1 : T_1), \ldots, (v_n : T_n)$$

If

$$\begin{aligned} & \mathsf{unify}(T_i, T) \text{ would succeed} \\ \wedge \quad & \forall j \neq i.\ \mathsf{unify}(T_j, T) \text{ would fail} \end{aligned}$$

Then

$$\begin{aligned} & s' := s[\mathsf{id} := v_i] \\ & \mathsf{unify}(T_i, T) \end{aligned}$$

Iterative symbol resolution

Function called with the set of all overloaded symbols.

```
let iterative_resolution (current: symbol stack) : unit
  let next : symbol stack = Stack.create() in
  while not (Stack.is_empty current) do (* here, the stack [next] is empty *)
    let progress = ref false in
    while not (Stack.is_empty current) do (* this loop fills [next] *)
      let sym = Stack.pop current in
      if is_unresolved sym then begin
        attempt_resolution sym;
        if is_unresolved sym
          then Stack.push next sym
          else progress := true;
      end;
    done;
    if not !progress then raise Stuck;
    Stack.swap next current;
  done
```

Soundness and completeness

Typechecking: ML-typechecking followed by iterated symbol resolution.

Theorem (Type soundness)

If typechecking succeeds, then the extracted program is well-typed in ML.

Theorem (Unambiguity)

If typechecking succeeds, then no other instantiation of the overloaded symbols leads to a well-typed extracted ML program.

Theorem (Completeness)

Typechecking succeeds if and only the program is iteratively solvable.

Complexity bounds

Let X denote the set of overloaded symbols. Define:

$N = \|\|X\|\|$	number of symbols
$M = \max_{x \in X} \ \|\mathcal{T}(x)\|$	max size of the type of a symbol
$J = \sum_{x \in X} \sum_{T \in \mathcal{I}(x)} \|T\|$	size of instance descriptions
$P = \sum_{t \,\text{subterm}} \|\mathcal{T}(t)\|$	size of the output typed tree

Theorem (Cost of typechecking)

$$O\left(\alpha(P) \cdot (P + J + MN^2)\right)$$

In practice, dependency chains generally have a bounded depth, hence the $O(N^2)$ can be replaced with just $O(N)$.

Conclusion

Contribution: a sound and complete, polynomial-time, multidirectional, static resolution algorithm for overloaded symbols.

Draft paper available on demand, including:

- overloading of data constructors and record field names
- instances with assumptions, e.g. matrix add depends on add
- handling a bound on the depth of nested assumptions
- typechecking of the motivating example

```
let demo (m:complex matrix) (n:complex matrix) =
  bigsum [i; 2*i] (fun d ->
    bigsum (int_range (-6) 7) (fun k ->
      3 * (e ^ (d * pi / 8)) * (m ^ (2*k^2)) * n))
```

- typechecking of realistic code: Aberth's root finding code
- benchmarking performance on worst-case input programs